

รายงานการประเมิน Knowledge Retrieval & Mapping Module

การทดลองเปรียบเทียบ Cross-lingual Generation Pipeline (5 Variants)
และการประเมินสามชั้น: Retrieval / Mapping / Generation

CyberCase Intelligence Framework — branch eval/crosslingual-generation
วันที่ 9 กรกฎาคม พ.ศ. 2569 | โมเดล: claude-haiku-4-5 | ชุดทดสอบ: สำนวนคดีจำลอง 45 ข้อ

1. ภาพรวมและที่มา

คำถามตั้งต้นของการทดลอง: pipeline ปัจจุบันรับ query ภาษาไทย แต่ context ที่ retrieve จากฐานความรู้ MITRE ATT&CK เป็นภาษาอังกฤษ — reasoning LLM จะสับสนหรือไม่ และการจัดเส้นทางภาษา (แปลก่อน / ไม่แปล / ยุบขั้นตอน) แบบใดให้คุณภาพต่อต้นทุนดีที่สุด

งานนี้จึงสร้างระบบประเมินครบสามชั้นของโมดูล โดยทุกชั้นวัดบน ground truth ชุดเดียวกัน ทำให้ชี้ได้ว่าปัญหาอยู่ชั้นใด:

ชั้น	คำถามที่ตอบ	Metric หลัก
Knowledge Retrieval	หลักฐานของแต่ละชั้นเหตุการณ์ถึงมือ LLM หรือไม่	step-coverage@k (แยก named/described)
Mapping (ส่ง backend)	ตาราง technique ที่กรอง noise แล้ว แม่นแค่ไหน	Precision/Recall/F1 เทียบ gold + threshold sweep
Generation	คำตอบสุดท้ายอ้าง technique ถูก ครบ และเป็นไทยดีหรือไม่	ID-F1 (partial credit), guard metrics, paired stats

2. สิ่งที่ทำและสิ่งที่เปลี่ยนแปลง

2.1 แกঁสูตร Recall (capped recall@k)

ปัญหาเดิม: dataset สร้าง gold จากการ traverse กราฟจาก seed ที่ degree สูง ทำให้บางข้อมี gold นับร้อยตัว ขณะที่ retriever คืนได้ $k=10 \Rightarrow \text{recall}$ ถูกบล็อกเพดานทางคณิตศาสตร์ (เช่น gold 180 ตัว คะแนนเต็มคือ $10/180 = 0.056$) ตัวเลขจึงไม่สะท้อนฝีมือ retriever

การแก้: เปลี่ยนตัวหารจาก $|gold|$ เป็น $\min(|gold|, k)$ — ถามว่า "k ช่องที่มี ใช้คุ้มแค่ไหน" และทำคู่กับการถอด LIMIT 20 ออกจาก group_techniques เพื่อให้เฉลยสมบูรณ์ (สองการแก้เป็นแพ็คเกจ: เฉลยข้อสตัย + สเกลที่ตีความได้)

2.2 Metric ใหม่: step-coverage@k สำหรับसानวนคดี

โจทย์จริงเป็นเหตุการณ์หลายชั้นเรียงเวลา การวัดแบบ flat recall บอกไม่ได้ว่าพลาดชั้นไหน จึงเปลี่ยน gold เป็นลำดับ attack_steps และวัดว่าแต่ละชั้นมีหลักฐานใน top-k หรือไม่ (เทียบเคียง Subtopic Recall — Zhai et al., SIGIR 2003 และแนวทาง nugget ของ TREC RAG 2024) พร้อมแยกประเภท cue: named (โจทย์ระบุชื่อเทคนิค เช่น "ใช้ SQL injection") กับ described (เล่าเฉพาะพฤติกรรม เช่น "ลบข้อมูลทั้งหมด") ซึ่งทดสอบความสามารถคนละแบบ

2.3 Dataset ใหม่: สำนวนคดีจำลอง 45 ข้อ

สร้างแบบกึ่งอัตโนมัติ: (1) สุ่ม kill-chain จากเทคนิคที่กลุ่มโจมตีจริงกลุ่มเดียวใช้ร่วมกัน (การันตี combo เกิดจริง, เฉพาะ tactic ที่เหยื่อสังเกตได้, กรองเทคนิคที่ MITRE ยกเลิกแล้ว 248 รายการ) (2) LLM ร่างสำนวนไทยเรียงเวลาแบบภาษาพนักงานสอบสวน โดยอ้างอิงสำนวนจริงจากข่าวคดีของ บช.สอท. (3) ระบบตรวจอัตโนมัติ (auto-flag) จับข้อผิด 2 แบบ: เฉลยไม่ตรงตัวอักษรกับโจทย์ และข้อ described ที่เผลอบอกชื่อเทคนิค (4) มนุษย์รื้อวก่อนใช้จริง — gold ต่อข้อ 3–7 เทคนิค จึงไม่เจอปัญหา recall เพดานอีก

2.4 การแก้คุณภาพข้อมูลที่พบระหว่างทาง

- Neo4j ไม่ได้เก็บ flag revoked/deprecated จาก STIX $\Rightarrow$ สร้าง blacklist จาก bundle v19 ในเครื่อง
- attack_id ซนกันข้ามประเภท (Mitigation เก่าใช้ ID เดียวกับ Technique เช่น T1064) $\Rightarrow$ แกँ lookup ให้ล็อก label เฉพาะ Technique/Subtechnique

- eval สายเก่า (chain.py) แปล query ก่อน retrieve แต่ production จริง (agent path) ไม่แปล — แก่ harness ให้ mirror agent path เป็น (decomposer □ quota retrieval)

2.5 Harness การทดลอง 3 เฟส (ทำซ้ำ/รันต่อได้ทุกเฟส)

retrieve — freeze retrieval หนึ่งครั้งต่อข้อ ทุก variant ใช้ context เดียวกัน (แยกผลของ generation ออกจาก retrieval) □ generate — รัน 5 variants บน context แซ่แข็ง □ score / score-mapping / score-retrieval — ให้คะแนนสามชั้น + สถิติ paired (bootstrap 95% CI + Wilcoxon signed-rank เทียบ baseline)

3. วิธีที่ใช้ปัจจุบัน: การเปรียบเทียบ 5 เส้นทางภาษา

Variant	เส้นทาง	LLM calls	หมายเหตุ
A (baseline)	query ไทยตรง □ reason อังกฤษ □ แปลไทย	2	= production agent ปัจจุบัน
B	เหมือน A + แบนคำแปลอังกฤษของ query ใน prompt	3	ทดสอบว่าการเห็นคำถามสองภาษาช่วยไหม
C	call เดียวจบ: คิดภายใน เขียนคำตอบไทยเลย	1	= fast mode ที่มีอยู่ ทดสอบเลื่อนเป็น default
D	เหมือน A แต่ขั้นแปลใช้โมเดลถูก (Haiku)	2	ข้ามในรอบนี้ (reasoning เป็น Haiku อยู่แล้ว)
E (อ้างอิง)	query อังกฤษ □ ตอบอังกฤษ ไม่มีขั้นไทย	1	ไม่ใช่ตัวเลือกใช้งาน — ไว้วัดต้นทุนของชั้นภาษาไทย

ทุก variant ใช้โมเดลเดียวกัน (claude-haiku-4-5) และ context ชุดเดียวกัน — ตัวแปรเดียวที่ต่างคือรูปร่างของ pipeline

4. ผลลัพธ์ (45 ข้อ × 4 variants)

4.1 Generation — คุณภาพคำตอบสุดท้าย

Metric	A (baseline)	B (+แปล query)	C (call เดียว)	E (EN อ้างอิง)
ID-F1	0.479	0.473	0.468	0.391
ID Precision	0.371	0.369	0.366	0.274
ID Recall	0.718	0.699	0.688	0.721
Tactic F1	0.811	0.812	0.810	0.759
Step coverage (named)	0.935	0.923	0.935	0.935
Step coverage (described)	0.504	0.507	0.476	0.517
โครงสร้าง 4 หัวข้อครบ	1.000	1.000	1.000	1.000
สัดส่วนอักษรไทย	0.886	0.891	0.835	0.000
ID รอดข้ามขั้นแปล	1.000	1.000	—	—
Latency (วินาที)	29.5	28.3	12.6	8.2
Output tokens เฉลี่ย	3,460	3,383	1,945	778

สถิติแบบจับคู่ (paired) เทียบ baseline A บน ID-F1:

Variant	Δ เฉลี่ย	95% CI (bootstrap)	Wilcoxon p	ตีความ
B	−0.006	[−0.023, +0.010]	0.84	ไม่ต่างจาก A — จ่ายเพิ่ม 1 call โดยไม่ได้อะไร
C	−0.011	[−0.029, +0.007]	0.24	ไม่ต่างจาก A — แต่เร็วกว่า 2.3 เท่า
E	−0.087	[−0.118, −0.058]	< 0.0001	แยกอย่างมีนัยสำคัญ

4.2 Knowledge Retrieval — step-coverage@k (เส้นทาง production)

k	Coverage รวม	named cue	described cue
5	0.441	0.704	0.283
10	0.550	0.835	0.367
15	0.601	0.885	0.419
20	0.601 (ตัน)	0.885	0.419

การตั้งตั้งแต่ k=15 หมายความว่า ขั้นตอนการค้นแบบ described ที่พลาด ไม่ได้อยู่ใน candidate เลย — เป็นปัญหาการค้นหา (candidate generation) ไม่ใช่การจัดอันดับ

4.3 Mapping — ตารางที่ส่งไป backend เทียบ gold

Source	Precision	Recall	F1	IDs/ข้อ
Raw retrieval (ไม่กรอง — แนวคิดแรก)	0.079	0.788	0.138	64.4

Source	Precision	Recall	F1	IDs/ข้อ
ตาราง mapping ผ่านตัวกรอง (คำตอบ A)	0.395	0.696	0.490	8.1
ตาราง mapping ผ่านตัวกรอง (คำตอบ C)	0.404	0.696	0.497	7.9

Threshold sweep (ค่า config ปัจจุบัน 0.62):

Threshold	Precision	Recall	F1	IDs/ข้อ
0.00 – 0.60 (ผลนี้ทั้งช่วง)	0.238	0.707	0.351	12.9
0.70	0.445	0.696	0.530	7.2

5. ข้อสรุปและข้อเสนอ

- (1) เส้นทางภาษาแบบไม่มีผลต่อคุณภาพ — จำนวน call ต่างหากที่มีผลต่อต้นทุน: variant C (call เดียว) เสมอ baseline ทางสถิติ แต่ latency ลดจาก 29.5 $\square$ 12.6 วินาที และ token ลด 44% $\square$ เสนอเลื่อน fast mode เป็นค่าเริ่มต้นของ production
- (2) ชั้นภาษาไทยไม่ได้ทำคุณภาพหล่น: variant E (อังกฤษล้วน) ที่คาดว่าเป็นเพดาน กลับแยกกว่าอย่างมีนัยสำคัญ (precision 0.274 เทียบ 0.371) — คำตอบอังกฤษอ้างเทคนิคเกินจริงมากกว่า และการแปลไม่เคยทำ ATT&CK ID สูญหายหรือออกเพิ่มเลย (id_survival = 1.0 ทั้ง 90 คำตอบ)
- (3) การแนบคำแปลของ query ไม่ช่วย: variant B ไม่ต่างจาก A (p=0.84) — ไม่ควรเพิ่มชั้นแปล query เข้า pipeline
- (4) คอขวดของทั้งระบบอยู่ที่ retrieval ของพฤติกรรมแบบ described: หาเจอเพียง 0.419 และต้นตั้งแต่ k=15 ทุก variant — งานปรับปรุงที่คุ้มที่สุดคือฝั่ง retrieval เช่น ให้ query decomposer แปลงพฤติกรรมเป็นภาษาเทคนิคก่อนค้น
- (5) ชั้น mapping พิสูจน์คุณค่า: กรอง noise จาก 64 IDs เหลือ 8 IDs ต่อข้อ precision ดีขึ้น 5 เท่า และ threshold sweep ชี้ว่าควรปรับ MITRE_TABLE_SCORE_THRESHOLD จาก 0.62 $\square$ 0.70 (F1 0.490 $\square$ 0.530) — รอยืนยันบน dataset ที่รีวิวแล้ว
- (6) พบประเด็นควรตรวจต่อ: answer recall (0.72) สูงกว่า retrieval coverage (0.60) แปลว่าบางคำตอบมาจากความรู้ภายในโมเดล ไม่ใช่จาก context — เป็นเหตุผลให้พัฒนา faithfulness judge เป็นงานถัดไป

6. ข้อจำกัดของผลรอบนี้

- โจทย์ 45 ข้อยังไม่ผ่านการรีวิวโดยผู้เชี่ยวชาญ (15 ข้อมี auto-flag) — ตัวเลขอาจขยับเมื่อรีวิวเสร็จ
- ใช้โมเดล Haiku ทั้งหมด — variant D ยังไม่ถูกทดสอบ และผลอาจต่างเมื่อใช้ Sonnet
- ยังไม่มีการวัด faithfulness (ความ ground ของคำอธิบาย) — วัดเฉพาะความถูกต้องของ ID
- n=45 เพียงพอสำหรับ paired comparison แต่ผลย่อยราย category ยังมีความไม่แน่นอนสูง

7. การทำซ้ำ (Reproducibility)

โค้ดทั้งหมดอยู่บน branch eval/crosslingual-generation (12 commits) — ไฟล์หลัก:
evaluation/crosslingual_generation_benchmark.py (harness), make_incident_dataset.py (ชุดข้อสอบ),
attack_id_metrics.py + retriever_metrics.py (metrics), ผลลัพธ์จัดใน data/gen_bench/ และรายงานใน results/
คำสั่งรันซ้ำ: `python -m RAG.GraphRAG.evaluation.crosslingual_generation_benchmark --phase all --reasoning-model claude-haiku-4-5` (ทุกเฟส resume ได้ ค่าใช้จ่ายรอบเต็มประมาณ \$3)